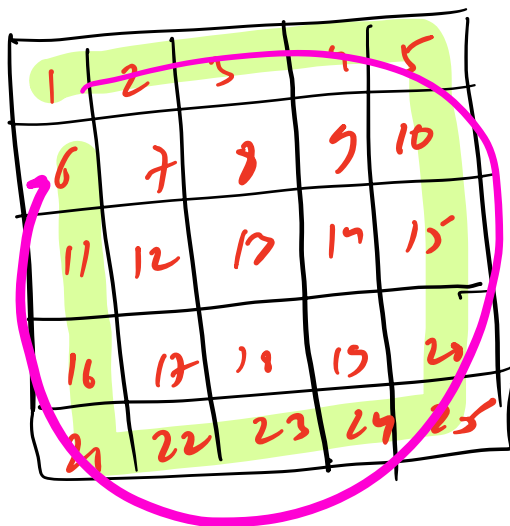


Q

Given a $N \times N$ matrix.

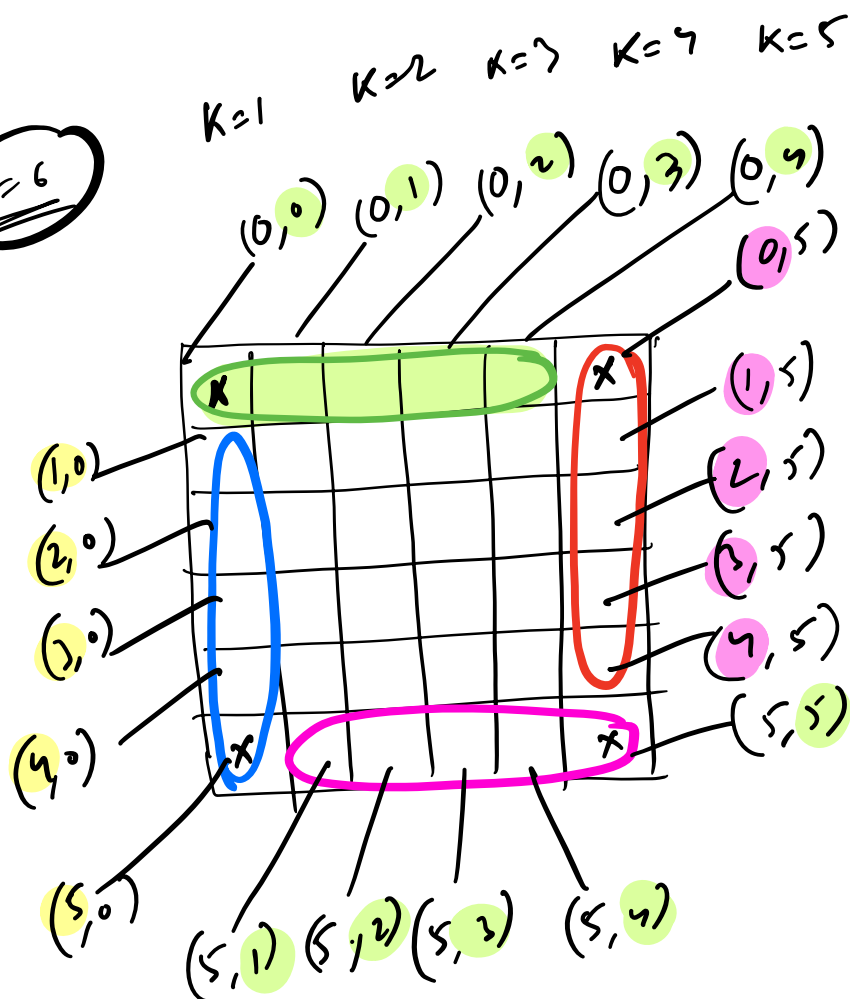
Print the boundary in clockwise fashion!

I/P



O/P $\rightarrow$ 1, 2, 3, 4, 5, 10, 15, 20, 25, 24, 23, 22, 21, 16, 11, 6

$N=6$



- STEPS:**
1. print 5 elements to right
 2. down
 3. left
 4. up
- $N-1$

$i=0, j=0;$

```
f( k=1; k<N; k++) {  
    print(A[i][j]);  
    j++;  
}
```

$\rightarrow N$
+

```
f( k=1; k<N; k++) {  
    print(A[i][j]);  
    i++;  
}
```

$\rightarrow N$
+

```
f( k=1; k<N; k++) {  
    print(A[i][j]);  
    j--;  
}
```

$\rightarrow N$
+

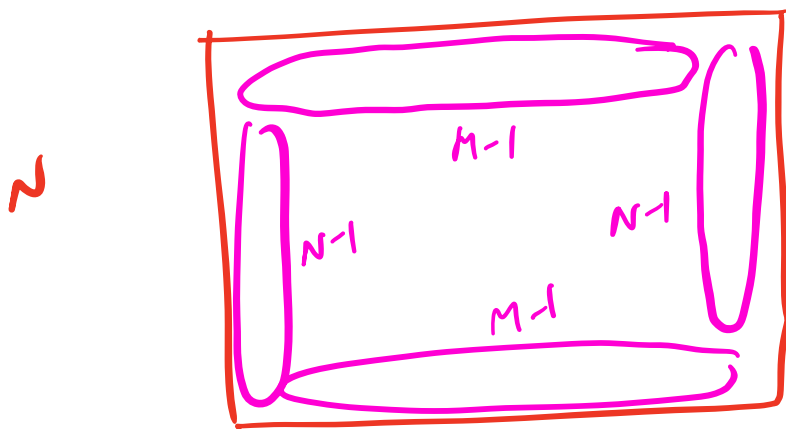
```
f( k=1; k<N; k++) {  
    print(A[i][j]);  
    i--;  
}
```

$\rightarrow N$

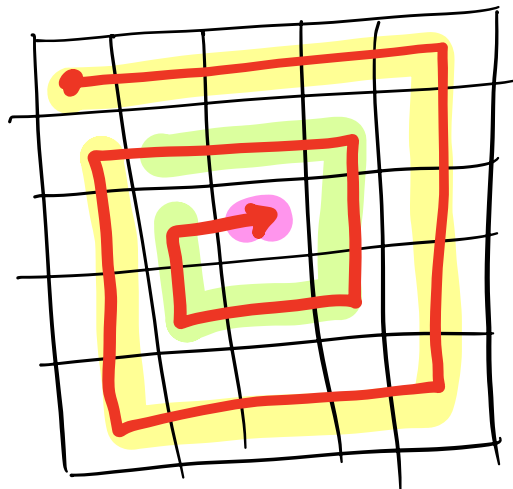
$T = O(N)$ ✓

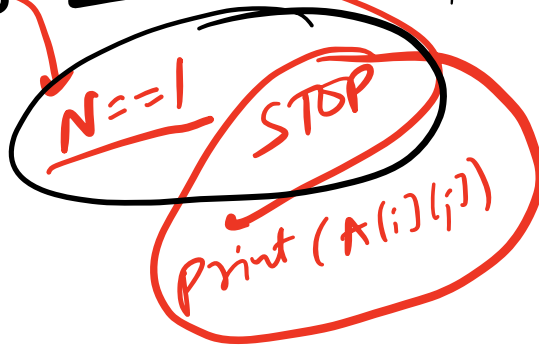
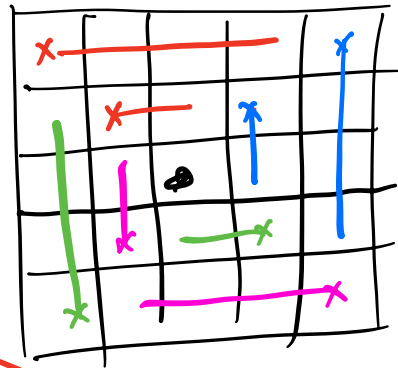
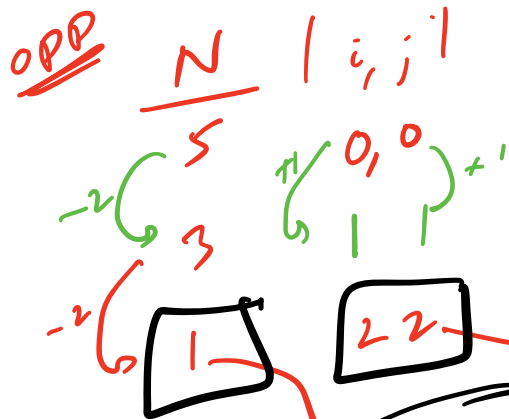
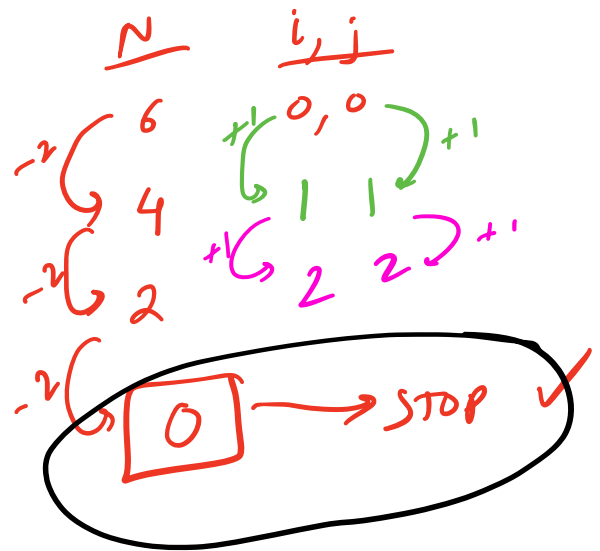
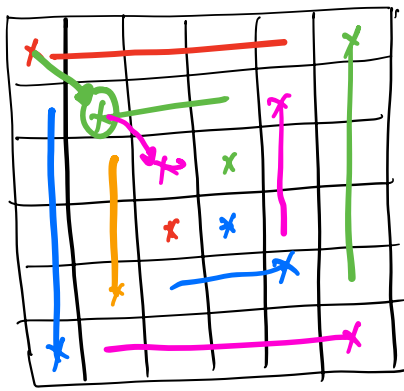
$i, j \rightarrow 0, 0$

Q SAME Q for rect. $N \times M$



Q Given a $N \times N$ matrix.
Print it in **SPIRAL** fashion!





```
// N i=0, j=0  
while ( N > 1 ) {
```

```
    f ( k=1; k < N; k++ ) {  
        print ( A[i][j] );  
        j++;  
    }
```

```
    f ( k=1; k < N; k++ ) {  
        print ( A[i][j] );  
        i++;  
    }
```

```
    f ( k=1; k < N; k++ ) {  
        print ( A[i][j] );  
        j--;  
    }
```

```
    f ( k=1; k < N; k++ ) {  
        print ( A[i][j] );  
        i--;  
    }
```

```
    N = N - 2;  
    i++, j++;
```

```
    if ( N == 1 )  
        print ( A[i][j] );
```

~~TC = $O(N^2)$~~

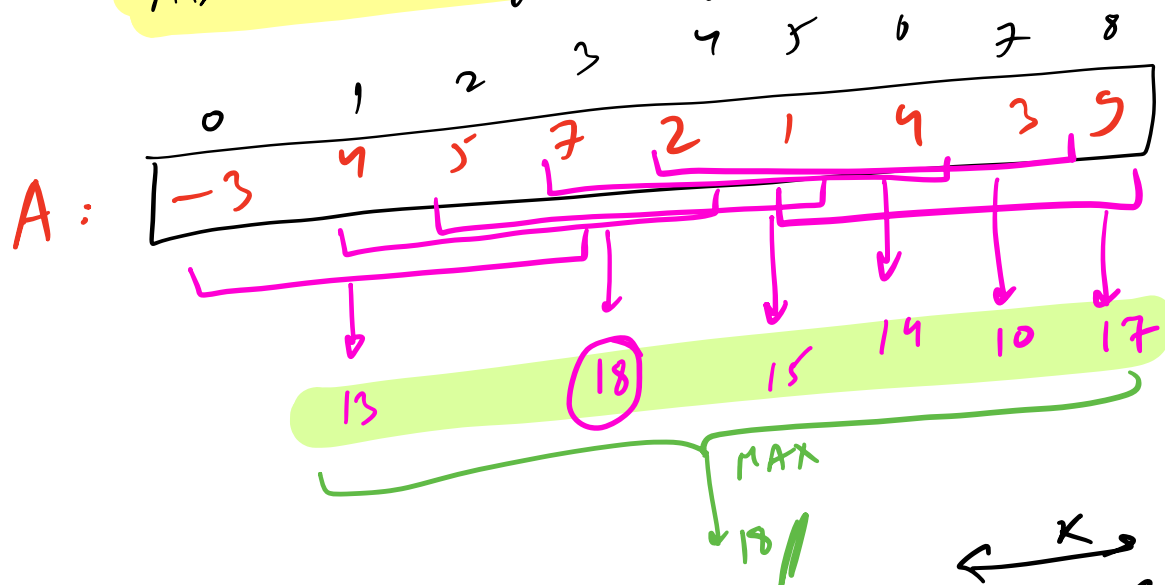
~~SC = $O(1)$~~

Q

Given N elements, return the
Maximum Subarray Sum of len = K !

$K=4$
 $N=9$

$K=4$
 $N=9$



1) BF

$L=0, R=K-1; m(-\infty);$

while ($R < N$) {

// $[L, R] \rightarrow$ Sub-Array of size K

$S=0;$

{ $i=L; i \leq R; i++$ }

$S += A[i];$

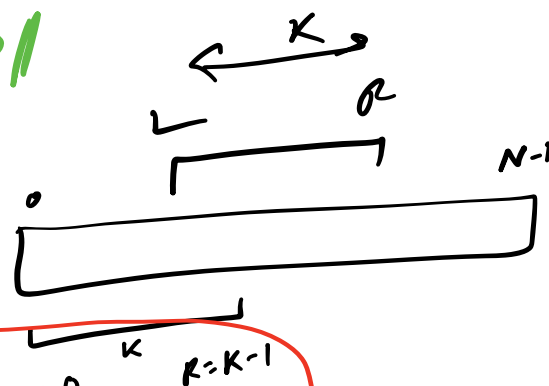
}

$m = \max(m, S);$

$L++, R++;$

}

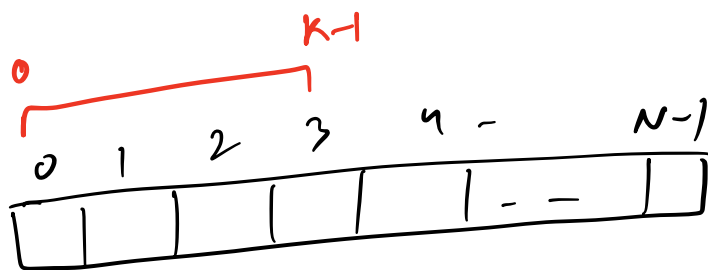
ret m ;



#SAs of size K

$O(K)$

$TC = O((N-K+1) \times K)$
 $TC = O((N-K)K)$



SA	L	R
1	0	k-1
2	1	k
3	2	k+1
...	...	...
$N-k+1$	$N-k$	$N-1$

Green arrows indicate increments of 1 in the SA and L columns. A green box highlights the value $N-k+1$ in the SA column.

SA of len k in A of size $N = N-k+1$

$$TC = \frac{(N-k) \cdot k}{2}$$

$k=1$



$TC = O(N)$

$N-k = k$
 $N = 2k$
 $k = N/2$



$(N-N/2) \cdot (N/2)$
 $N^2/4$

$TC = O(N^2)$

$k=N$



$TC = O(N)$

II

Prefix Sums

1. Build the $ps[]$ $\rightarrow O(N)$

2. $L = 0, R = K - 1; m = -\infty;$ $\rightarrow N - K$

while ($R < N$) {
 // $[L, R] \rightarrow$ Sub-Array of size k

$S = ps[R] - ps[L - 1];$ $\rightarrow L == 0 : 0$

$m = \max(m, S);$ $\rightarrow O(1)$

$L++, R++;$

}

return $m;$

$TC = O(N + (N - K) \times 1)$

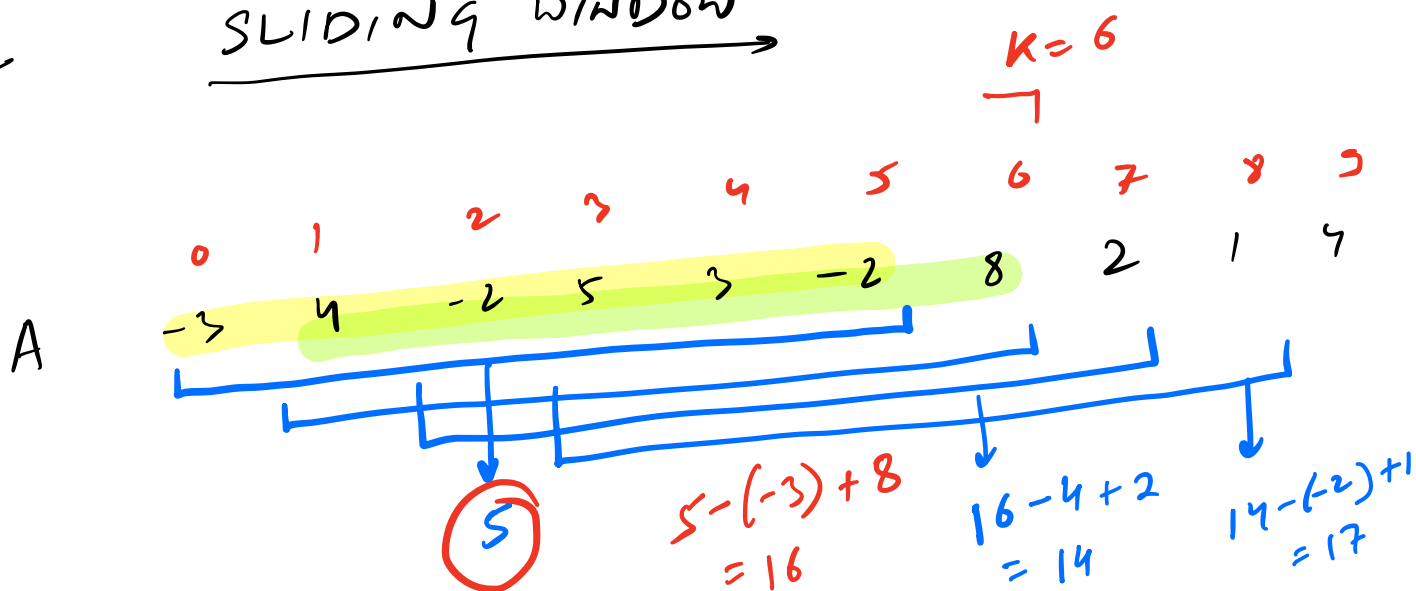
$TC = O(N)$

$SC = O(N)$

$\rightarrow O(1)$

III

SLIDING WINDOW



L	R	
0	5	→ Iterate and find the sum
1	6	→ $sum - A[0] + A[6]$
2	7	→ $sum - A[1] + A[7]$
3	8	→ $sum - A[2] + A[8]$
		→ $sum - A[L-1] + A[R]$
L	R	

*** $sum = sum - A[L-1] + A[R]$**

$S = 0;$

$\{ (i = 0; i < K; i++) \{ L = 1 \quad K = K$
 $S += A[i]; \quad \rightarrow K$

$\}$

$ANS = S;$

$L = 1, R = K;$

$\text{while} (R < N) \{$

$// [L, R]$

$S = S - A[L-1] + A[R];$

$ANS = \text{MAX}(ANS, S);$

$L++, R++;$

$\}$

$\text{ret } ANS;$

$N - K$

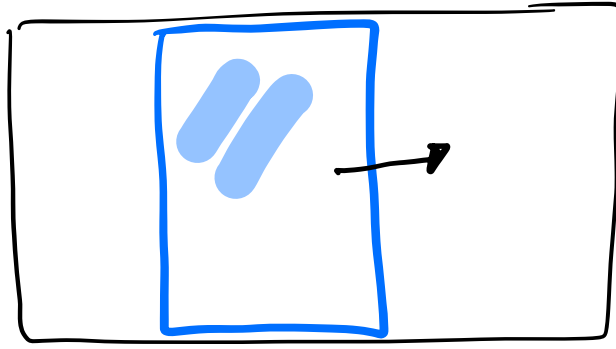
$\rightarrow O(1)$

$TC = K + N - K$

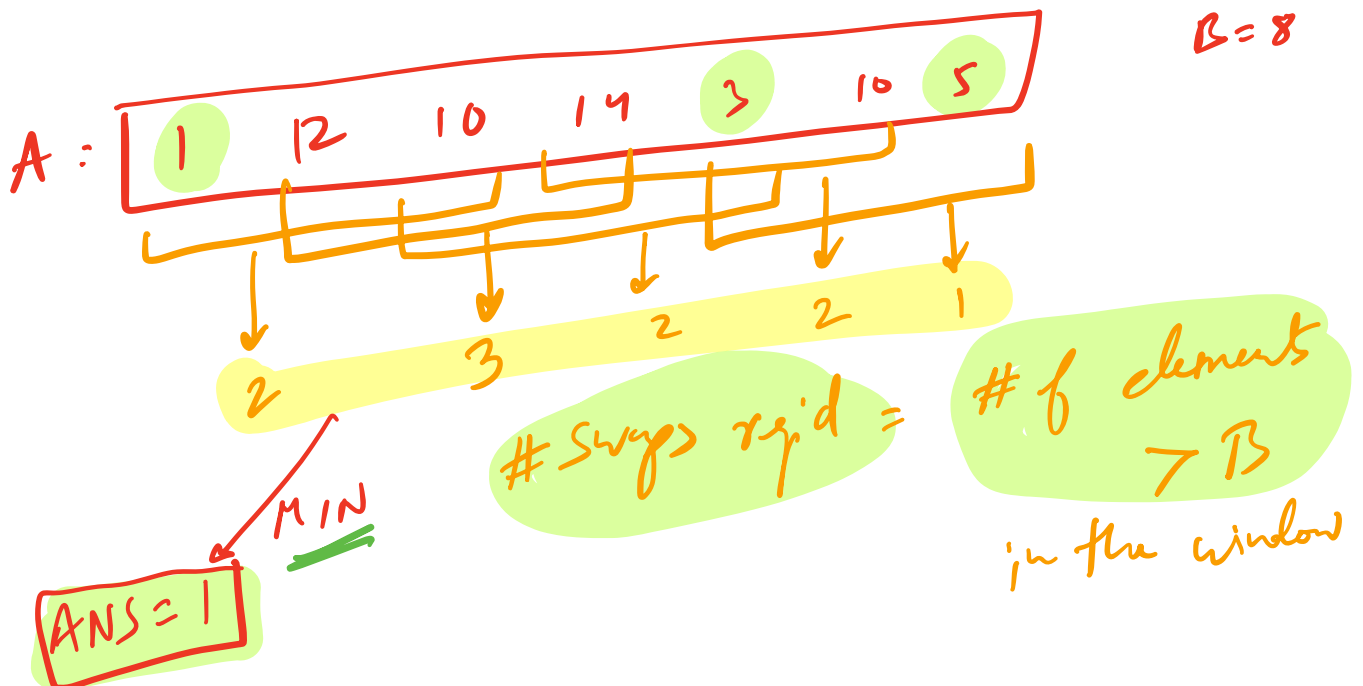
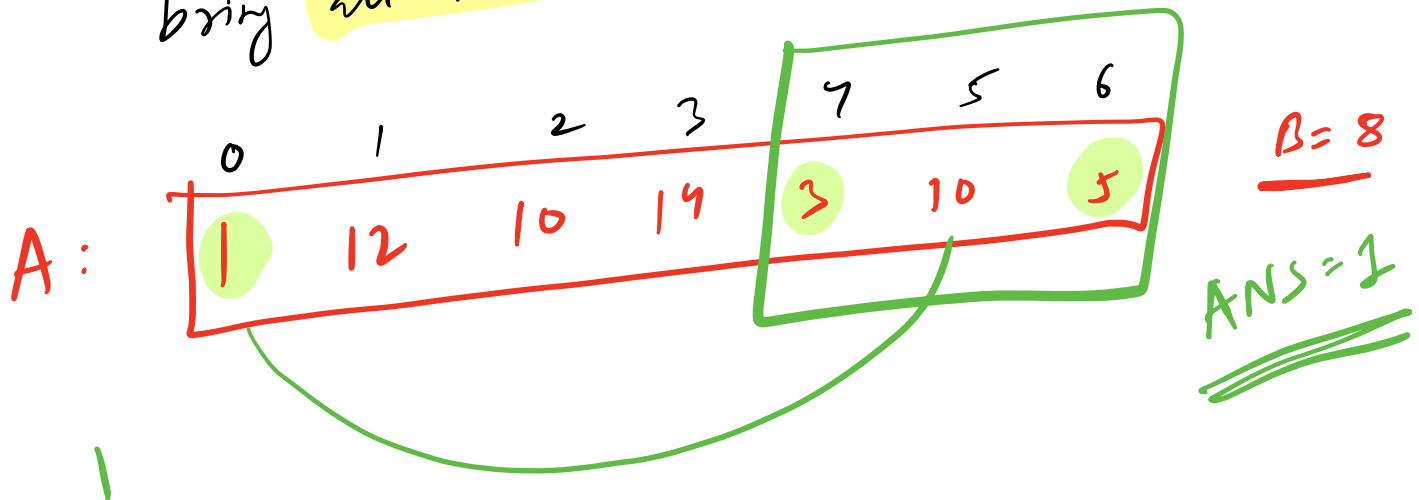
$TC = O(N)$

$SC = O(1)$

WINDOW size is FIXED in Sliding Window!



Q Given an array A & an int B.
find the MIN no. of swaps required to
bring all the elements $\leq B$ together!



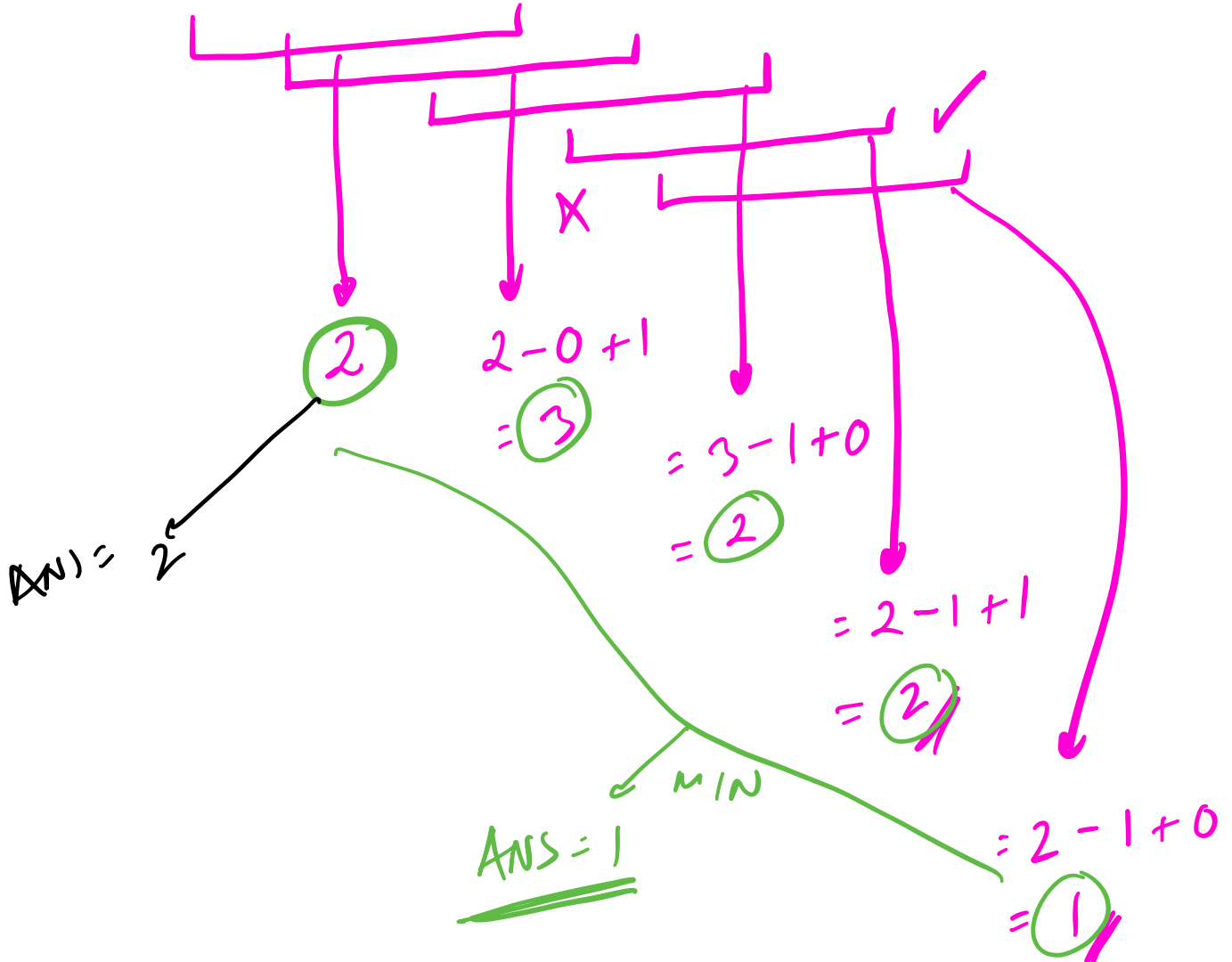
Obj: If windows of len K , find the # of element

$B=8$

$7B$

$K=3$

A: [1 12 10 14 3 10 5]



$K = 0;$

```
f (i = 0 → N-1) {  
    if (A[i] ≤ B) {  
        K++;  
    }  
}
```

→ N

cnt = 0;

```
f (i = 0; i < K; i++) {  
    if (A[i] > B) {  
        cnt++;  
    }  
}
```

→ K

ANS = cnt;

L = 1; R = K;

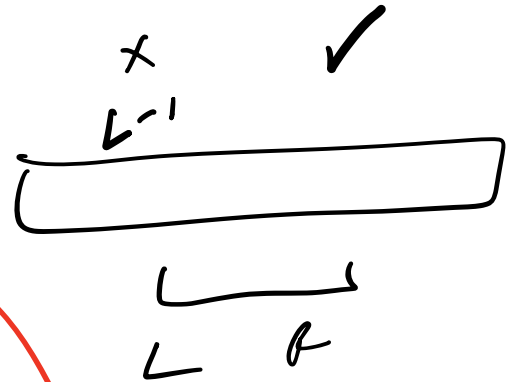
while (R < N) {

```
    if (A[L-1] > B) {  
        cnt--;
```

```
    }  
    if (A[R] > B) {  
        cnt++;
```

}

ANS = min(ANS, cnt);



→ N-K

→ O(1)

L++, R++;

}

return ANS

~~$N + N$~~

$2N$

~~$TC = O(N)$~~

~~$SC = O(1)$~~

)